

SIMATS ENGINEERING
ASSESSMENT TOOL 2 (APPLICATION BASED PROBLEM SOLVING)

COURSE CODE:CSA0927

COURSE NAME: Programming in JAVA

COURSE OUTCOME COVERED

CO5. *Construct database-driven web applications by implementing JDBC connectivity to perform SQL operations such as insert, update, and delete using different statement types. (BL3, BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 2

Total Marks:100

Annexure A

APPLICATION BASED PROBLEM SOLVING

Code of Conduct:

*I N.Navya Sri Vasavi(192411204) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Vasavi

APPLICATION BASED PROBLEM SOLVING

1: Smart Library Book Registration System (Statement + PreparedStatement)

A college library is digitizing its book inventory. The librarian needs a Java application that allows new books to be added while preventing duplicate ISBN numbers from being stored.

Database Table: Books

BookID Title Author ISBN AvailableCopies

Tasks:

1. Establish JDBC connectivity with MySQL.
2. Insert five book records using PreparedStatement.
3. Before insertion, verify whether the ISBN already exists using a Statement.
4. Display an appropriate success or duplicate message.

Problem-solving focus: Choosing between Statement and PreparedStatement for validation and insertion.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Analysis & Solution Design	20%	Accurately analyzes the application scenario and designs a complete JDBC-based solution with appropriate SQL operations.	Correctly analyzes the scenario with minor gaps in the solution design.	Partially understands the scenario and proposes an incomplete solution.	Unable to identify the problem requirements or design a suitable solution.
JDBC Connectivity & Statement Usage	25%	Correctly establishes database connectivity and effectively uses the appropriate JDBC statement type (Statement, PreparedStatement, or CallableStatement).	Connection and statement implementation are mostly correct with minor errors.	JDBC connectivity is partially implemented; statement usage is inconsistent.	Fails to establish database connectivity or use JDBC statements correctly.

SQL Operations & Data Integrity	20%	Implements INSERT, UPDATE, and DELETE operations accurately while maintaining data integrity and proper parameter handling.	SQL operations are correct with minor logical or syntax issues.	Performs basic SQL operations with noticeable errors in data handling.	SQL operations are incorrect or fail to modify the database as required.
Program Logic & Exception Handling	20%	Demonstrates strong logical reasoning, validates inputs, and handles exceptions effectively using appropriate techniques.	Good program logic with basic input validation and exception handling.	Limited logical flow with minimal validation or error handling.	Poor program logic with no meaningful exception handling.
Code Quality & Output Verification	15%	Produces clean, modular, well-documented code and correctly verifies database results through meaningful output.	Code is organized with minor documentation or output verification gaps.	Code structure is inconsistent; output verification is incomplete.	Code is poorly structured with incorrect or missing output verification.

CODE :

```
import java.sql.Connection;
import
java.sql.DriverManager;
import
java.sql.PreparedStatement;
import java.sql.ResultSet;
import
```

```
java.sql.Statement;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        // MySQL connection details
```

```
        String url =
```

```
        "jdbc:mysql://localhost:3306/testdb"; String
```

```
        user = "root";
```

```
        String password = "Gani@2006";
```

```
        try {
```

```
            // 1. Establish JDBC connection
```

```
            Connection con = DriverManager.getConnection(url, user, password);
```

```
            System.out.println("MySQL Connected Successfully!");
```

```
            // 2. Create Books table
```

```
String createTable = "CREATE TABLE IF NOT EXISTS Books (" +  
    "BookID INT PRIMARY KEY, " +  
    "Title VARCHAR(100), " +  
    "Author VARCHAR(100), " +  
    "ISBN VARCHAR(30) UNIQUE, " +  
    "AvailableCopies INT)";
```

```
stmt.executeUpdate(createTable);
```

```
System.out.println("Books table ready!");
```

```
// 3. Five book records
```

```
String[][] books = {  
    {"1", "Java Programming", "James Gosling", "9781001", "5"},  
    {"2", "Database Systems", "Raghu Ramakrishnan", "9781002", "3"},  
    {"3", "Computer Networks", "Andrew Tanenbaum", "9781003", "4"},  
    {"4", "Data Structures", "Mark Allen Weiss", "9781004", "6"},  
    {"5", "Operating Systems", "Abraham Silberschatz", "9781005", "2"}  
};
```

```
// 4. PreparedStatement for
```

```
insertion String insertSQL =  
    "INSERT INTO Books " +  
    "(BookID, Title, Author, ISBN, AvailableCopies) " +  
    "VALUES (?, ?, ?, ?, ?)";
```

```
PreparedStatement pstmt = con.prepareStatement(insertSQL);
```

```
// 5. Process each book
```

```
for (String[] book : books) {
```

```
    int bookID = Integer.parseInt(book[0]);  
    String title = book[1];  
    String author =  
    book[2]; String isbn =  
    book[3];  
    int copies = Integer.parseInt(book[4]);
```

```
    // Check ISBN using Statement
```

```
    String checkSQL =  
        "SELECT ISBN FROM Books WHERE ISBN = '" + isbn + "'";
```

```
    ResultSet rs =
```

```
    stmt.executeQuery(checkSQL); if
```

```
    (rs.next()) {
```

```
        // Duplicate ISBN found
```

```
        System.out.println(  
            "Duplicate ISBN: " + isbn  
            + " - Book not inserted."  
        );
```

```
    } else {
```

```
        // Insert using
```

```
        PreparedStatement
```

```
        pstmt.setInt(1, bookID);
```

```
        pstmt.setString(2, title);
```

```
pstmt.setInt(5, copies);  
  
pstmt.executeUpdate();  
  
System.out.println(  
    "Book inserted successfully: " + title  
);  
}
```

```

        rs.close();
    }

    // 6. Display all books
    System.out.println("\n===== BOOK DETAILS =====");

    ResultSet rs = stmt.executeQuery("SELECT * FROM

Books"); while (rs.next()) {

        System.out.println(
            "Book ID: " + rs.getInt("BookID")
            + ", Title: " + rs.getString("Title")
            +
            ", Author: " + rs.getString("Author")
            + ", ISBN: " + rs.getString("ISBN")
            +
            ", Available Copies: " + rs.getInt("AvailableCopies")
        );
    }

    // 7. Close resources
    rs.close();
    pstmt.close();
    stmt.close();
    con.close();

    System.out.println("\nConnection

closed.");

} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());
}
}
}

```

Local MySQL - x

File Edit View Query Database Server Tools Scripting Help

Navigator: Query 1 x SQLAdditions

SCHMAS

Filter objects

sys

Tables

sys_config

Columns

variable

value

set_time

set_by

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

Query 1

1 USE testdb;

2

3 SELECT * FROM Books;

Limit to 1000 rows

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

BookID	Title	Author	ISBN	AvailableCopies
1	Java Programming	James Gosling	9781001	5
2	Database Systems	Raghu Ramakrishnan	9781002	3
3	Computer Networks	Andrew Tanenbaum	9781003	4
4	Data Structures	Mark Allen Weiss	9781004	6
5	Operating Systems	Abraham Silberschatz	9781005	2

Books 5 x

Administration Schemas

Information

Schema: sys

Output

Action Output

#	Time	Action	Message	Duration / Fetch
6	14:25:29	USE testdb	0 row(s) affected	0.016 sec
7	14:25:29	SELECT * FROM student LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
8	14:28:04	USE testdb	0 row(s) affected	0.000 sec
9	14:28:04	SELECT * FROM student LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
10	14:29:01	USE testdb	0 row(s) affected	0.000 sec
11	14:29:01	SELECT * FROM Books LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Object Info Session